Comparing Language Models, Embeddings and Frozen Features

Experiments on WikiText-2 and IMDb

1. Introduction

This study addresses three connected questions. How do trigram, RNN, LSTM and causal Transformer language models compare on WikiText-2? Does changing the input embeddings improve an otherwise matched LSTM? And does a lower language-model perplexity lead to more useful features for IMDb sentiment classification? We compare predictive performance, computational cost, learning dynamics, generated text and downstream transfer, while separating observed results from possible explanations.

2. Data, preprocessing and model design

WikiText-2 raw-v1 provides official train, validation and test splits. We use document-aware word-level tokenisation, preserve case and punctuation, normalise WikiText formatting, and keep level-1 document boundaries. Each document is wrapped with <bos> and <eos>; a context never crosses documents. A 20,000-entry vocabulary (including <pad>, <unk>, <bos>, <eos>) is built from training text only. The processed splits contain 629/60/62 documents and 2,032,891/212,129/238,414 encoded tokens (train/validation/test). Their OOV rates are 5.07%/7.57%/8.10%. Neural targets are formed in windows up to 64 positions; padding targets are ignored, and <bos> is not scored. Every Part I model is evaluated on the same 212,069 validation and 238,352 test targets.

The RNN and LSTM each have one recurrent layer, 128-dimensional trainable embeddings and 256 hidden units. Neither uses dropout. The Transformer has two decoder blocks, 128-dimensional hidden states, four attention heads, a 512-dimensional feed-forward layer, learned positional embeddings and dropout 0.1. Its causal attention was implemented with PyTorch tensor operations rather than nn.MultiheadAttention. We also implemented RNN and LSTM cells ourselves and checked them against PyTorch; the benchmark models use nn.RNN and nn.LSTM for training.

The embedding experiment uses a separate LSTM with 100-dimensional inputs so that all three embedding settings have the same dimension. For sentiment classification, we split the official IMDb training set into 20,000 training and 5,000 validation reviews (stratified, seed 42), keeping the official 25,000-review test set separate. We remove HTML breaks, apply the WikiText tokenizer and vocabulary, and truncate reviews at 768 tokens. The recorded IMDb OOV rate is 7.14%. Reviews are processed in independent 64-token windows; the language models are not fine-tuned on IMDb.

3. Evaluation protocol and fairness

We train the neural language models with Adam (learning rate 0.001), batch size 16, gradient clipping at 1.0 and seed 42. Training is capped at 15 epochs, with early stopping based on validation cross-entropy (patience 3, minimum improvement 0.001). We evaluate the checkpoint with the lowest validation loss. Perplexity is exp(cross-entropy), where cross-entropy is measured in nats per predicted token. The training times in the main tables add up the recorded training steps only; they exclude validation, preprocessing and separate embedding pretraining. Throughput is measured when scoring the same test targets. The trigram runs on CPU, while the neural models use MPS, so their speeds should not be compared as if the hardware were identical. For IMDb, we fit StandardScaler on the training split and choose logistic-regression C from {0.01, 0.1, 1, 10, 100} using validation macro-F1.

4. Architecture comparison: perplexity and convergence

Our trigram combines unigram, bigram and trigram probabilities (additive alpha = 10; tau2 = tau3 = 320, chosen using validation data). The neural models share the corpus, word vocabulary, prediction targets and core optimisation settings, but have access to up to 64 previous positions rather than two. They also differ in depth, hidden width, dropout and completed epochs. Consequently, this is a comparison of trained systems under a common task, not a controlled experiment that isolates architecture alone.

Model	Test PPL	Parameters	Training (s)	Test tokens/s
Trigram	258.38	—	1.6	844,996
RNN	169.41	7.80M	615.5	99,931
LSTM	150.15	8.10M	672.5	119,519
Transformer	134.21	5.54M	778.8	153,416

On the same 238,352 test targets, the Transformer achieves the lowest perplexity (134.21), followed by LSTM (150.15), RNN (169.41) and trigram (258.38). Its test PPL is 10.61% below LSTM's with fewer trainable parameters (5.54M versus 8.10M). This parameter difference partly reflects the output projection: the recurrent models map 256-dimensional states to 20,000 vocabulary logits, while Transformer uses 128-dimensional states. Relative to the trigram, Transformer reduces test perplexity by about 48%, showing the benefit

of the richer context and learned sequence representations in this evaluated setup. LSTM also improves substantially over RNN, supporting the value of its gated memory for these 64-token windows. The same ranking appears on the validation set, so the central pattern is consistent across both held-out splits.

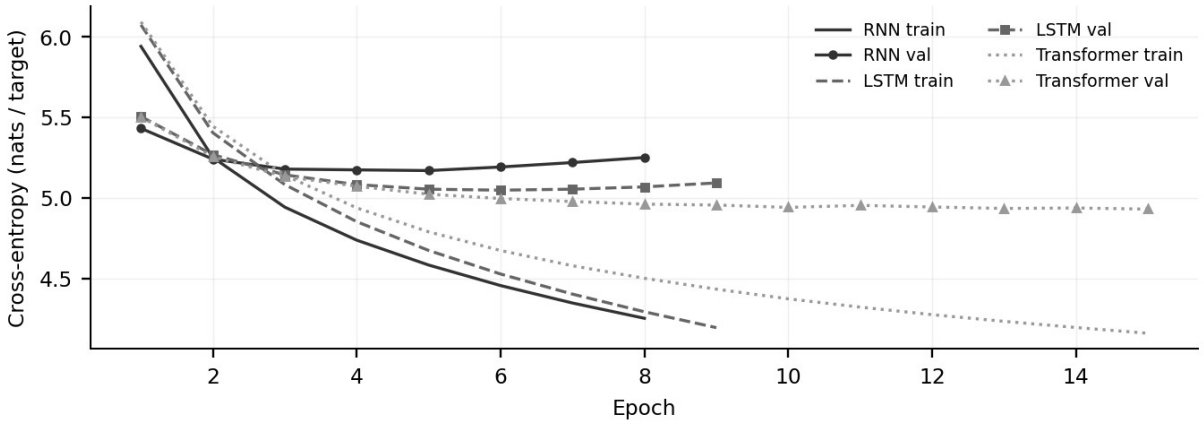


Figure 1. Recorded training and validation cross-entropy. Each model follows its own early-stopping run.

The learning curves show distinct training patterns. RNN reaches its best validation CE (5.1692) at epoch 5, but validation CE increases to 5.2501 by epoch 8 while training CE falls to 4.2528. LSTM behaves similarly: validation CE rises from 5.0476 at epoch 6 to 5.0924 at epoch 9 as training CE falls to 4.1947. The widening train–validation gap is consistent with overfitting under the chosen settings. Transformer improves more slowly, reaching its lowest validation CE (4.9299) at epoch 15, with minor fluctuations along the way. Unlike the recurrent models, it reaches the budget limit without a demonstrated validation plateau; we cannot determine whether further training would improve it.

The validation–training gap provides a useful diagnostic, but it is not itself a measure of out-of-domain generalisation. In these runs, the recurrent models begin to fit the training tokens increasingly well after their best validation checkpoints, whereas the Transformer continues to improve on held-out WikiText-2 through the final epoch. The shared 15-epoch cap therefore affects the models differently: early stopping curtails the recurrent runs, while the Transformer remains budget-limited. This distinction is important when interpreting both final perplexity and total wall-clock cost.

The epoch histories also show why a single final training loss would be misleading. The RNN's training objective keeps improving after the selected checkpoint, yet its validation loss moves in the opposite direction; evaluating the final epoch would therefore overstate the value of continued optimisation. The same applies to LSTM, although its lowest validation loss occurs one epoch later. Transformer presents a different uncertainty: it has not clearly reached its best attainable validation performance within fifteen epochs. Reporting the best checkpoints alongside the completed epoch counts keeps these differences visible. Our results compare the best validation-selected checkpoints rather than models trained for exactly the same number of epochs.

5. Architecture comparison: efficiency and generation

Computational efficiency depends on whether the model scores an existing sequence or generates new tokens. During teacher-forced test scoring, Transformer reaches 153,416 tokens/s, compared with 119,519 for LSTM and 99,931 for RNN. Its measured training time is lower per completed epoch (about 51.9 s versus 74.7–76.9 s for the recurrent models), although its 15 epochs make the total longer. Autoregressive generation reverses the ranking: approximately 440 tokens/s for LSTM, 347 for RNN and 203 for Transformer. Our Transformer recomputes the context at each generation step without a KV cache, so parallel scoring speed does not translate directly to token-by-token decoding speed. The trigram was measured on CPU rather than MPS; its throughput is included as a baseline, not as a hardware-controlled speed comparison.

The throughput measurements also separate parallel sequence scoring from iterative text generation. During evaluation, the entire input window is known, allowing the Transformer to compute attention across positions in parallel. During sampling, however, only one new token becomes available at a time. Repeatedly recomputing its full context makes this particular implementation slower to generate than the recurrent models, which advance a compact recurrent state. This is an implementation-level trade-off, not evidence that Transformer decoding must always be slower.

5.1 Common-prompt generation

We generate text from three fixed prompts: “The history of”, “In the city of” and “The first”. Every run uses seed 42, temperature 1.0, top-k 20 and up to 40 new tokens. We also save samples with <unk> excluded from sampling. Keeping these settings fixed helps us compare the examples, although a few random samples cannot tell us which model is generally the most fluent.

The samples illustrate why perplexity alone does not capture generation quality. From “In the city of”, the trigram produces repeated <unk> tokens, and LSTM also emits unknown words. RNN generates plausible local phrases, including “United States Marine Corps” and “Royal Navy”, but switches topics unexpectedly. Transformer begins “Toronto, the city was a city of Galveston”: individual phrases

look fluent, yet the location shift and repeated “city” weaken coherence. These are observations from fixed examples, not evidence that any model consistently writes better text.

Excluding <unk> makes some outputs easier to read, but it only changes how tokens are sampled; the language model and its test perplexity stay the same. The trigram’s two-word context is one possible reason for its frequent topic changes. The neural models also work with short, 64-token contexts. More prompts and repeated generations would be needed before making a general claim about writing quality.

5.2 What the architecture comparison supports

Holding the corpus, vocabulary, scored targets, core optimisation settings and generation protocol consistent makes the model-level trade-offs directly observable. In our tests, Transformer attains the lowest perplexity with fewer parameters and the highest teacher-forced evaluation throughput, whereas LSTM is faster for incremental generation in the implemented setup. These results offer a practical comparison of predictive quality, checkpoint footprint and inference behavior on the same workload. Architecture, width, regularisation and completed training epochs are not individually matched, so the measured advantage belongs to the complete Transformer configuration rather than to attention alone.

The parameter and timing breakdown makes these observations useful for implementation decisions. In a word-level model, the output projection is costly: the recurrent models project 256-dimensional hidden states to 20,000 vocabulary logits, compared with 128-dimensional Transformer states. The Transformer also takes less time per training epoch, although its longer run increases total training time. Meanwhile, the recurrent models' efficient state updates help explain their generation-speed advantage over our Transformer implementation without KV caching. Together, the comparisons identify concrete performance and engineering trade-offs; isolating one architectural component would be a separate experiment.

6. Embedding study: controlled factors and remaining confounds

In Part II, all runs use a one-layer LSTM with 256 hidden units, 100-dimensional inputs, identical WikiText-2 splits, 64-token windows and the same training procedure. We compare three practical embedding pipelines: randomly initialised vectors learned jointly with the language model, Skip-gram vectors trained on WikiText-2 and then frozen, and public GloVe vectors also kept frozen. Every LSTM trains its remaining weights from scratch. This is a valid comparison of the embedding strategies actually used: the two frozen pretrained pipelines can be evaluated directly against learning embeddings within the language-model objective. Because both the source of vectors and their trainability differ, the result reflects the combined pipeline choice rather than the isolated effect of either factor.

7. Embedding ablation: results and interpretation

Embedding	Test PPL	Best epoch	LM train (s)	Trainable params
Random, trainable	153.57	6	640.4	7.51M
Skip-gram, frozen	137.24	8	726.0	5.51M
GloVe, frozen	137.34	8	782.6	5.51M

The random, trainable baseline gives 153.57 test PPL, compared with 137.24 for frozen Skip-gram and 137.34 for frozen GloVe—improvements of approximately 10.6%. The validation ranking agrees. Skip-gram uses only our training corpus, while GloVe draws on larger external text; their test perplexities differ by only 0.10, which does not establish a reliable ranking from a single run. The randomly initialized model learns a 20,000 × 100 embedding matrix (2 million trainable parameters); both frozen settings remove those parameters from optimisation. These results support the performance of the tested embedding pipelines, not a standalone causal claim that pretraining always beats learned random embeddings.

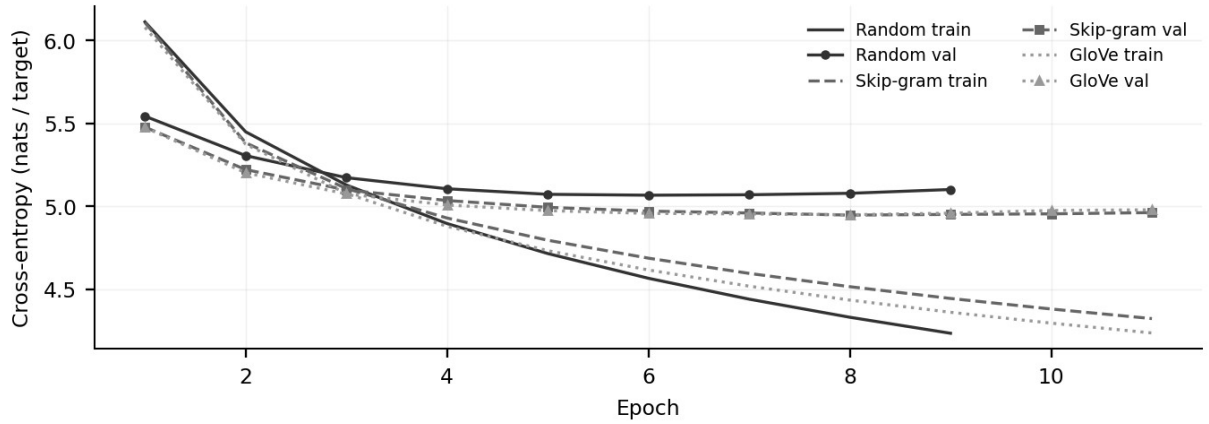


Figure 2. Per-epoch cross-entropy for the three 100-dimensional embedding settings.

All three training losses decrease smoothly, without obvious optimisation instability. Random embeddings attain their best validation loss at epoch 6; Skip-gram and GloVe do so at epoch 8. Validation loss then rises slightly while training loss continues down, suggesting overfitting rather than failure to fit the corpus. The pretrained settings achieve better validation minima, but need more epochs to reach them. Recorded mean training time per epoch is around 71.2 s (Random), 66.0 s (Skip-gram) and 71.1 s (GloVe). Single-run timing differences are too noisy to attribute an efficiency advantage to a particular embedding source.

We trained Skip-gram using gensim Word2Vec on WikiText-2 training text (window 5, five negative samples, five epochs, seed 42). This added 23.28 s beyond LSTM training and covered all 19,996 ordinary vocabulary entries. For GloVe, 12,954 entries matched exactly, 6,784 matched after lowercasing and 258 used seeded random vectors (98.71% coverage). Lowercasing improves coverage but can collapse distinctions between capitalised forms in the frozen embeddings. Generated samples also qualify the perplexity result: Skip-gram repeats “the city of the city”, whereas GloVe begins with “St. Petersburg” before drifting. The results demonstrate better average next-token prediction, but not a consistent improvement in long-form generation quality. A random-frozen control and trainable pretrained variants would help disentangle initialization and freezing.

Vocabulary coverage adds another qualification to the embedding comparison. Self-trained Skip-gram has a vector for every ordinary project token, whereas GloVe needs a lowercase fallback for thousands of entries and a random fallback for the remaining 258. As a result, the GloVe pipeline mixes pretrained and random inputs, while the Skip-gram pipeline uses vectors learned entirely from the project training split. Their nearly identical test perplexities are therefore notable, but they do not isolate the value of external pretraining data.

An important result is that the self-trained Skip-gram pipeline performs nearly as well as public GloVe (137.24 versus 137.34 test PPL), even though Skip-gram uses only the project's WikiText-2 training text. It also produces vectors for every one of the 19,996 ordinary vocabulary entries, avoiding the lowercase and random fallbacks required for GloVe. This gives a practical advantage when vocabulary coverage and control over the pretraining corpus matter. Taken together, the results show that both fixed-vector approaches are effective in this LSTM setup and that a relatively lightweight, corpus-specific Skip-gram model is a competitive alternative to public embeddings.

8. Frozen representations on IMDb

For IMDb, we freeze the Part I LSTM and Transformer and use their hidden states as features. No language-model weights are updated on reviews. We truncate each review at 768 tokens and process it as independent chunks of at most 64 tokens. Because each chunk is encoded separately, the model cannot directly pass information between chunks. We try several ways of turning the token states into one vector per review: mean pooling, token-weighted mean, mean plus max, and mean pooling with overlapping windows (stride 32 or 16). For LSTM, we also test the average of the final state from each chunk.

All representations use the same IMDb split and logistic-regression pipeline. Standardisation is fitted on training data only, and C is selected by validation macro-F1 rather than test accuracy. Stride-32 overlap yields the highest validation accuracy for each encoder, so these are the two primary representations for the paired comparison. We also explored a 128-hidden-unit MLP on ordinary mean features: its validation accuracy was 71.32% with LSTM features and 71.06% with Transformer features, versus 71.00% and 71.16% for the corresponding linear probes. Since the gains were not consistent, we retain logistic regression as the common classifier for the main evaluation.

Representation	**Val acc.**	**Test acc.**	**Test macro F1**
LSTM final hidden state	58.06%	57.83%	57.83%
LSTM mean	71.00%	70.83%	70.83%

LSTM weighted mean	71.34%	71.82%	71.82%
LSTM, stride 32	72.16%	71.56%	71.56%
Transformer mean	71.16%	69.88%	69.87%
Transformer, stride 32	71.20%	70.60%	70.59%

Pooling has a larger observed effect than changing encoder family in several comparisons. With the same frozen LSTM and linear classifier, averaging final states across chunks yields only 57.83% test accuracy, while averaging token states reaches 70.83%—a 13.00 percentage-point gain without retraining the encoder. One plausible explanation is that pooling token states retains evidence scattered across a review, while the final state of each independently reset chunk may emphasize its ending. We did not measure token-level information retention directly. Adding max-pooled features also fails to help either encoder (LSTM 70.49% versus 70.83%; Transformer 69.74% versus 69.88%), showing that extra feature dimensions alone do not improve linear separability.

8.1 Window overlap and classifier capacity

Window stride	LSTM val / test	Transformer val / test
64: no overlap	71.00 / 70.83%	71.16 / 69.88%
32: 50% overlap	72.16 / 71.56%	71.20 / 70.60%
16: 75% overlap	71.32 / 71.34%	70.90 / 70.77%

Overlap gives boundary tokens representations with different surrounding context, but at additional extraction cost. For LSTM, stride 32 raises validation accuracy from 71.00% to 72.16% and test accuracy from 70.83% to 71.56% relative to non-overlapping mean pooling. Transformer changes from 71.16% to 71.20% on validation and from 69.88% to 70.60% on test. Stride 16 uses more overlapping windows yet lowers validation accuracy for both encoders; the extra context views do not guarantee a better pooled feature. A 128-unit MLP reaches 71.32% validation accuracy for LSTM mean features and 71.06% for Transformer mean features, offering no consistent gain over the linear probe.

Because each 64-token chunk is encoded independently, overlap does not create memory spanning the full review. Instead, it changes which nearby words can contribute to an individual token representation. Stride 32 offers a second view of many boundary tokens, while stride 16 repeats even more computation. The small and non-monotonic accuracy changes suggest that additional context views can help feature aggregation, but their value depends on the downstream classifier and the tested configuration.

Overlap changes how often a token contributes to the pooled representation and how much left or right context accompanies it within a window. It does not increase the underlying review length beyond the 768-token truncation limit, nor does it let a hidden state flow from one window into the next. A smaller stride therefore produces more computations and more partially redundant views, not necessarily new independent evidence. This distinction matters when interpreting the accuracy pattern: stride 32 improves the LSTM's validation result, but stride 16 does not. Without measuring feature-extraction time for every stride under the same conditions, we should not turn that pattern into a numerical claim about the best accuracy–cost trade-off.

Across the 11 representations, pooling decisions are more consequential than modest differences between matched frozen encoders. Mean, weighted mean and mean-plus-max stay relatively close, but LSTM final-state averaging performs much worse. Token weighting produces the highest LSTM test accuracy (71.82%), yet it is not the validation winner: its validation accuracy is 71.34%, below stride 32 at 72.16%. For Transformer, weighting lowers validation accuracy from 71.16% to 70.48% even though its test result is slightly higher than ordinary mean (70.42% versus 69.88%). Among five matched extraction settings—mean, weighted mean, mean+max and strides 32/16—LSTM is ahead on test every time. We nevertheless retain stride 32 as the selected representation for each encoder because the selection was made using validation macro-F1, not the test leaderboard.

9. Error analysis and cross-task discussion

The confusion matrices help explain the difference between the two selected models. On the balanced IMDb test set of 25,000 reviews, LSTM reaches 71.56% accuracy and Transformer 70.60%. Their performance on negative reviews is similar: LSTM correctly classifies 9,009 and Transformer 9,045. The larger difference is for positive reviews, where LSTM gets 8,881 correct but Transformer gets 8,605. In recall terms, Transformer is slightly better on negative reviews (72.36% versus 72.07%) and worse on positive ones (68.84% versus 71.05%).

Frozen encoder	True negative → predicted positive	True positive → predicted negative	Correct negative / positive
LSTM stride 32	3,491	3,619	9,009 / 8,881
Transformer stride 32	3,455	3,895	9,045 / 8,605

This difference amounts to 36 fewer negative-review errors but 276 more positive-review errors for Transformer. The two models also make different decisions on many of the same reviews. Both are correct on 14,737 examples and both wrong on 4,197; LSTM alone is correct on 3,153, while Transformer alone is correct on 2,913. Their errors are therefore not identical, even though their overall accuracies are close.

Individual disagreements illustrate possible sources of error without establishing a general failure mechanism. Both models call the negative review at IMDb index 2315 positive despite explicit negative wording; on index 2074, a negative review containing some positive terms, LSTM is correct and Transformer is not. On positive review 12876, which includes negation, the reverse occurs. These examples suggest that lexical cues and context can conflict, but the observations are not representative of every review. Correctness differs on 6,066 reviews: 3,153 favour LSTM and 2,913 favour Transformer. A paired McNemar comparison gives chi-square ≈ 9.5 ($p \approx 0.002$), consistent with a detectable difference on this test set, although the absolute accuracy difference is modest.

The confusion matrices also expose a useful class-specific pattern that aggregate accuracy alone would miss. The selected LSTM and Transformer identify similar numbers of negative reviews correctly, but LSTM correctly identifies 276 more positive reviews. Their 0.96-percentage-point accuracy gap is therefore concentrated mainly on the positive class. Together with the 6,066 reviews on which only one model is correct, this gives us concrete, complementary views of model performance: class-level error rates show where the gap occurs, while paired predictions identify the individual reviews behind it.

10. Main interpretation and limitations

Together, these experiments show that language-model performance and downstream representation quality are related but distinct. Transformer gives the strongest next-token predictions on WikiText-2; frozen Skip-gram and GloVe improve perplexity in the separate 100-dimensional LSTM comparison. Yet on IMDb, the feature pooling strategy often has a larger effect on sentiment accuracy than the choice between our frozen LSTM and Transformer encoders. The Part II checkpoints were never used as IMDb encoders, so the embedding gains cannot be claimed as transfer gains. In this particular frozen-probe pipeline, lower source-domain perplexity is not a sufficient predictor of downstream sentiment accuracy.

An important practical implication is that the best feature extractor cannot be chosen from language-model perplexity alone. Perplexity measures average next-token predictions over WikiText-2; the IMDb probe instead asks whether a pooled representation linearly separates positive and negative reviews. These objectives reward different properties, particularly when the encoder is frozen and cannot adapt to sentiment labels. The LSTM–Transformer reversal in the matched pooling comparisons therefore reflects a difference between evaluation tasks, rather than a contradiction in the language-model results.

Several factors constrain interpretation. The Part I architectures differ in depth, state width, dropout and number of training epochs. In Part II, embedding initialization and trainability are confounded, and public GloVe uses outside text. The word-level vocabulary creates unknown tokens; IMDb reviews are truncated to 768 tokens and encoded as independent 64-token chunks, limiting cross-chunk reasoning. Comparing 11 feature configurations using the same validation split may favour small chance differences. Matched-budget controls, additional seeds, and experiments with contextual aggregation across chunks would test whether these findings generalise.

There are two useful next steps that follow directly from these limitations. First, repeating the same training and probe pipeline across several seeds would show whether small differences—especially between pooling variants—are stable. Second, matched model-width and training-budget comparisons could narrow the set of explanations for the architecture results. For downstream transfer, a hierarchical review encoder or another method that combines information across chunks would test the effect of the current independent-window design.

11. Conclusion and reproducibility

Overall, better next-token prediction did not translate directly into better frozen sentiment features in our experiments. Transformer performs best on WikiText-2, but LSTM features give slightly higher IMDb accuracy under the matched pooling settings. Within the same LSTM, the choice of embedding changes perplexity, while the choice of pooling changes classification accuracy even more. Code, configurations, saved results and reproduction commands are available at https://github.com/leisure-cloud05/nlp-language-model-comparison. The language models and the locally trained Skip-gram vectors were developed for this study; public GloVe was used only in the embedding comparison. We used PyTorch, Hugging Face datasets, gensim, scikit-learn and matplotlib. ChatGPT assisted with some coding, debugging and knowledge explanation.

References

Merity, S., Xiong, C., Bradbury, J., & Socher, R. (2016). Pointer Sentinel Mixture Models. WikiText-2 dataset.

Maas, A. L., et al. (2011). Learning Word Vectors for Sentiment Analysis. IMDb dataset.

Pennington, J., Socher, R., & Manning, C. (2014). GloVe: Global Vectors for Word Representation.

Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient Estimation of Word Representations in Vector Space.

Software: PyTorch; Hugging Face datasets; gensim; scikit-learn; NumPy; matplotlib.

Appendix A. Full frozen-probe results

Representation	Best C	Validation acc.	Test acc.	Test macro F1
lstm	1	58.06%	57.83%	57.83%
lstm_mean	0.01	71.00%	70.83%	70.83%
lstm_mean_weighted	10	71.34%	71.82%	71.82%
lstm_mean_max	0.01	70.30%	70.49%	70.49%
lstm_overlap	0.1	72.16%	71.56%	71.56%
lstm_stride16	0.01	71.32%	71.34%	71.33%
transformer	10	71.16%	69.88%	69.87%
transformer_mean_weighted	100	70.48%	70.42%	70.41%
transformer_mean_max	10	70.82%	69.74%	69.73%
transformer_overlap	0.1	71.20%	70.60%	70.59%
transformer_stride16	10	70.90%	70.77%	70.76%

Appendix B. Selected IMDb confusion matrices

The matrices show raw counts on the balanced official IMDb test split (12,500 reviews per class). Both classifiers use the validation-selected stride-32 representation.

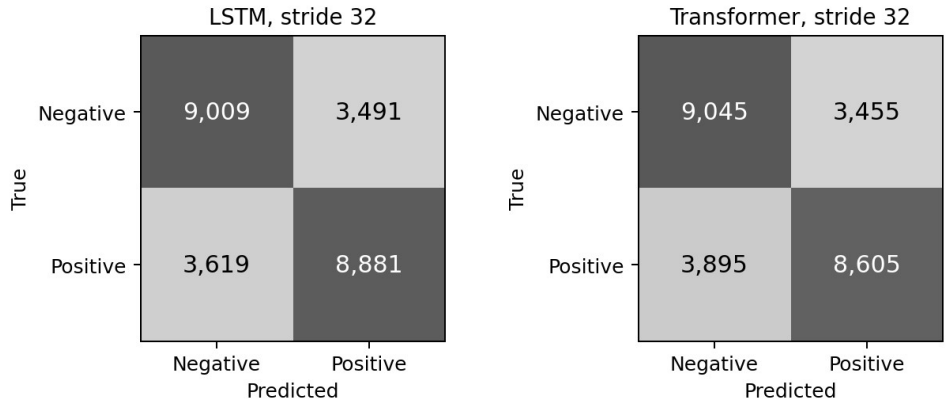


Figure B1. Confusion matrices for the frozen LSTM and Transformer probes. The main difference is on positive reviews: Transformer predicts 276 more positives as negative.